FOUNDATIONS OF WEB APP PENTESTING

1.3 | DEMO: REQUEST-RESPONSE CYCLE



Browser Requests • Network Tab • Rendering

HAROON SHAHID

FOUNDATIONS OF WEB APP PENTESTING

1.3 DEMO: REQUEST-RESPONSE CYCLE

WHAT THIS DEMO IS ACTUALLY SHOWING

This lecture is NOT teaching tools yet.

It's showing:

- How browser talks to server
- What actually happens when you open a website
- Where a pentester fits in

BASIC FLOW

When you open a website:

1. Browser (client) sends request
2. Server processes it
3. Server sends response
4. Browser shows (renders) it

REAL IMPORTANT TRUTH

Opening ONE website ≠ ONE request

It sends **MANY requests**

Example:

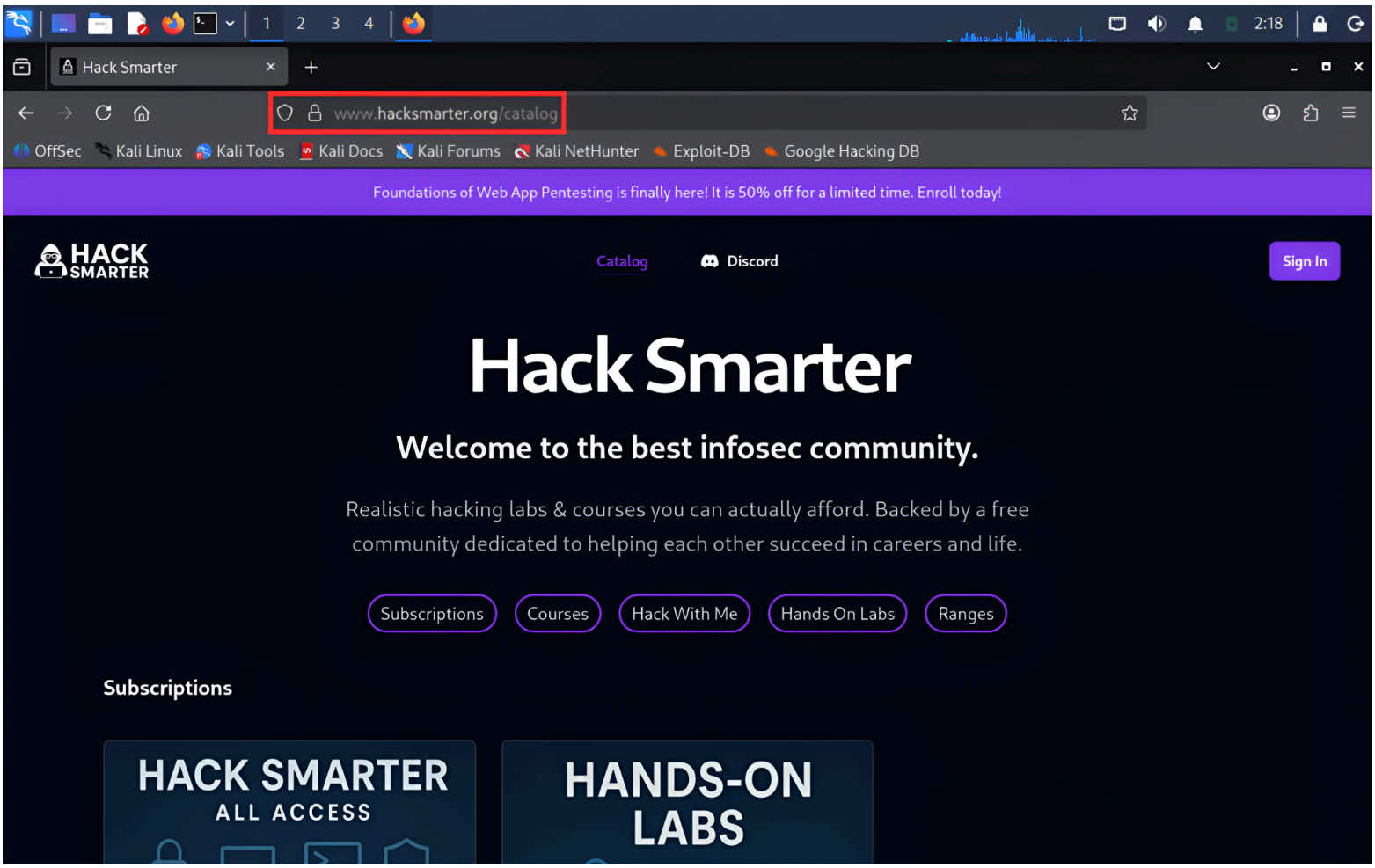
- HTML
- CSS
- JS
- Images
- Fonts

VIEWING REQUESTS IN BROWSER

(F12 METHOD)

Let’s visit a website for example hacksmarter.org/catalog.

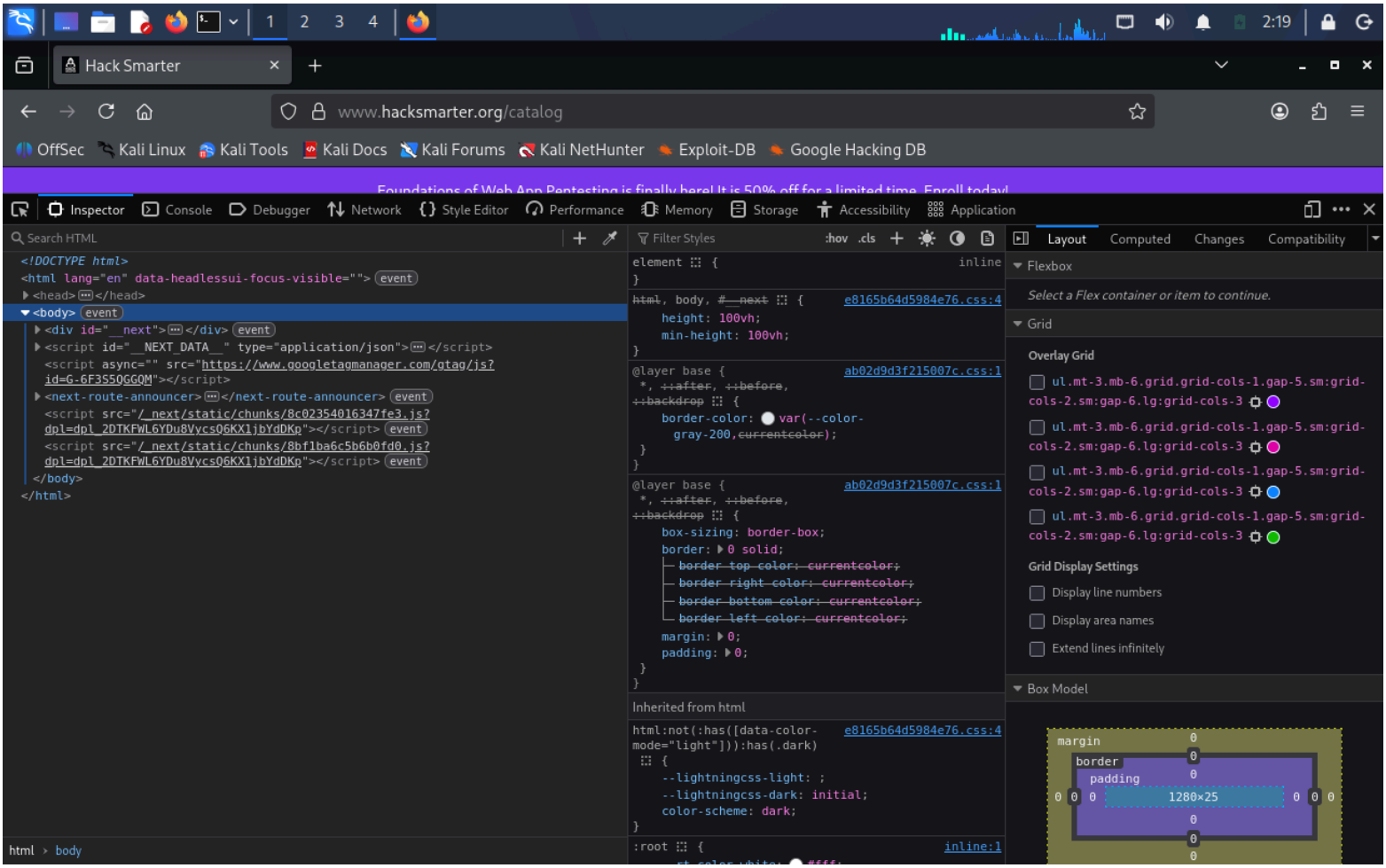
The place where I learned this course from.



Step 1: Open Developer Tools

Press:

F12

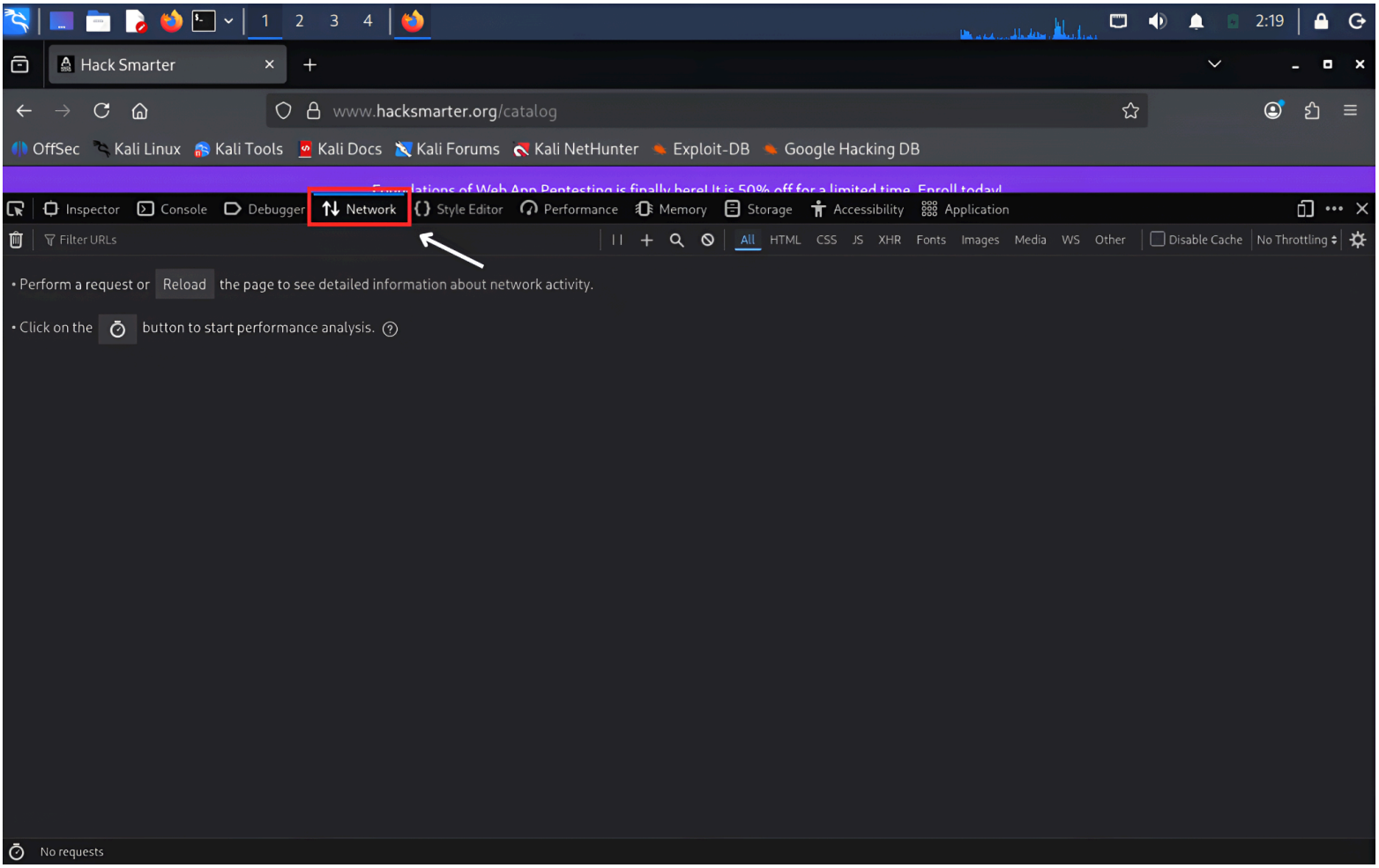


This opens **Developer Tools**

Step 2: Go to "Network" Tab

This shows:

- All requests
- All responses

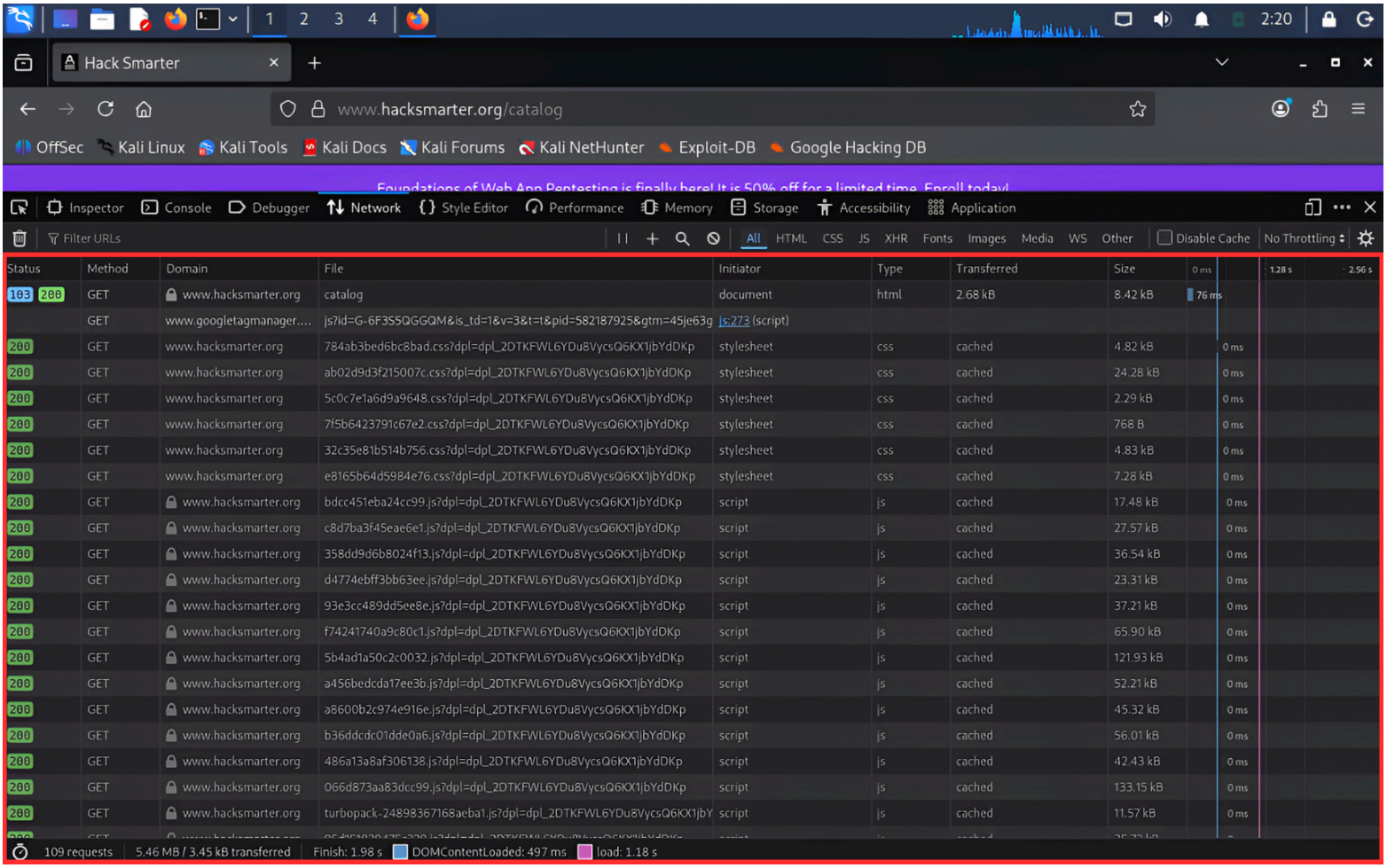


Step 3: Refresh the Page

Press:

`Ctrl + R`

Now you will see **a LOT of requests**



KEY LEARNING

One page load = **dozens of requests**

If you ignore this → you will be a weak pentester.

UNDERSTANDING ONE REQUEST

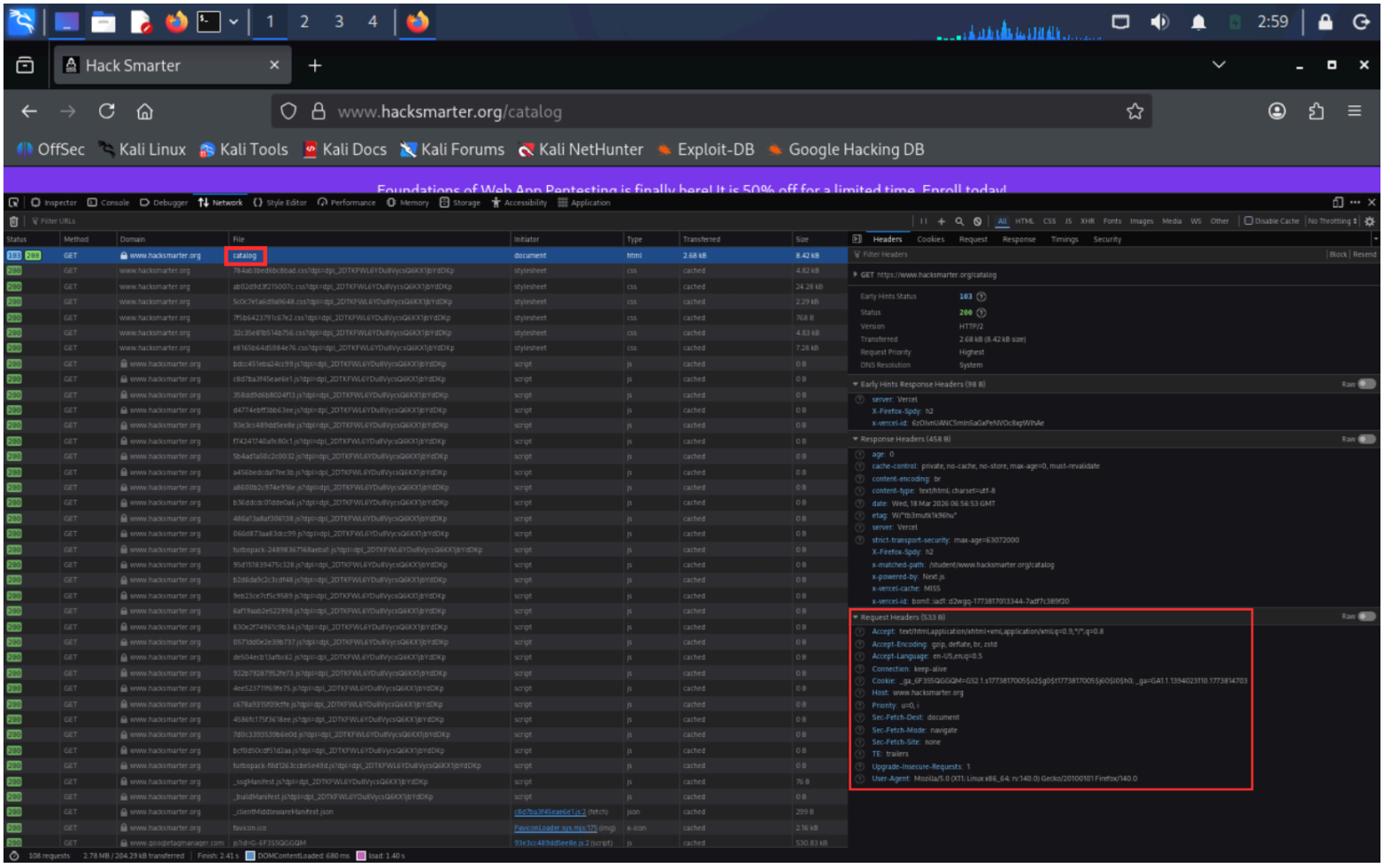
(CATALOG EXAMPLE)

Click any request (like `/catalog`)

REQUEST (CLIENT → SERVER)

Browser says:

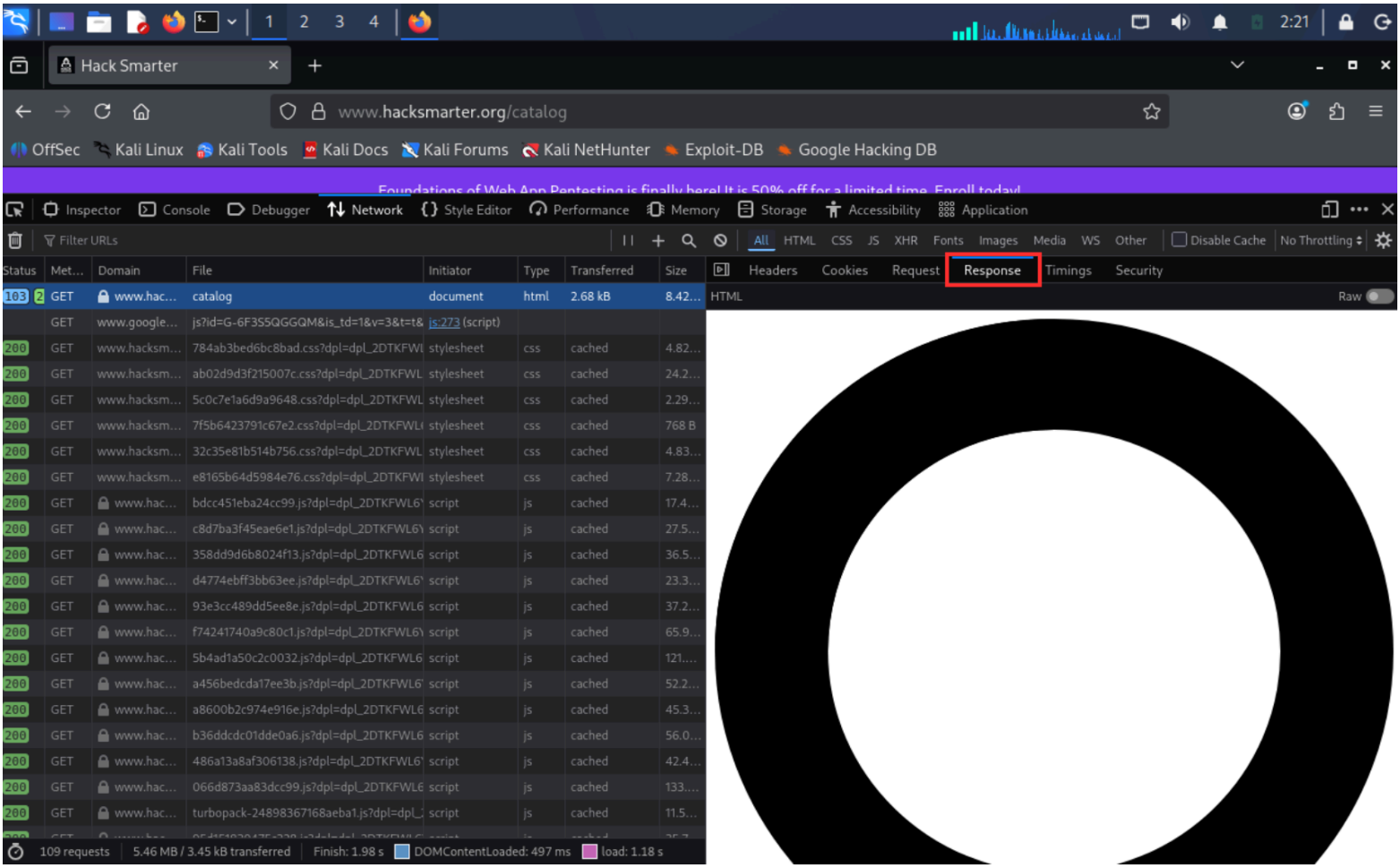
Give me `/catalog`



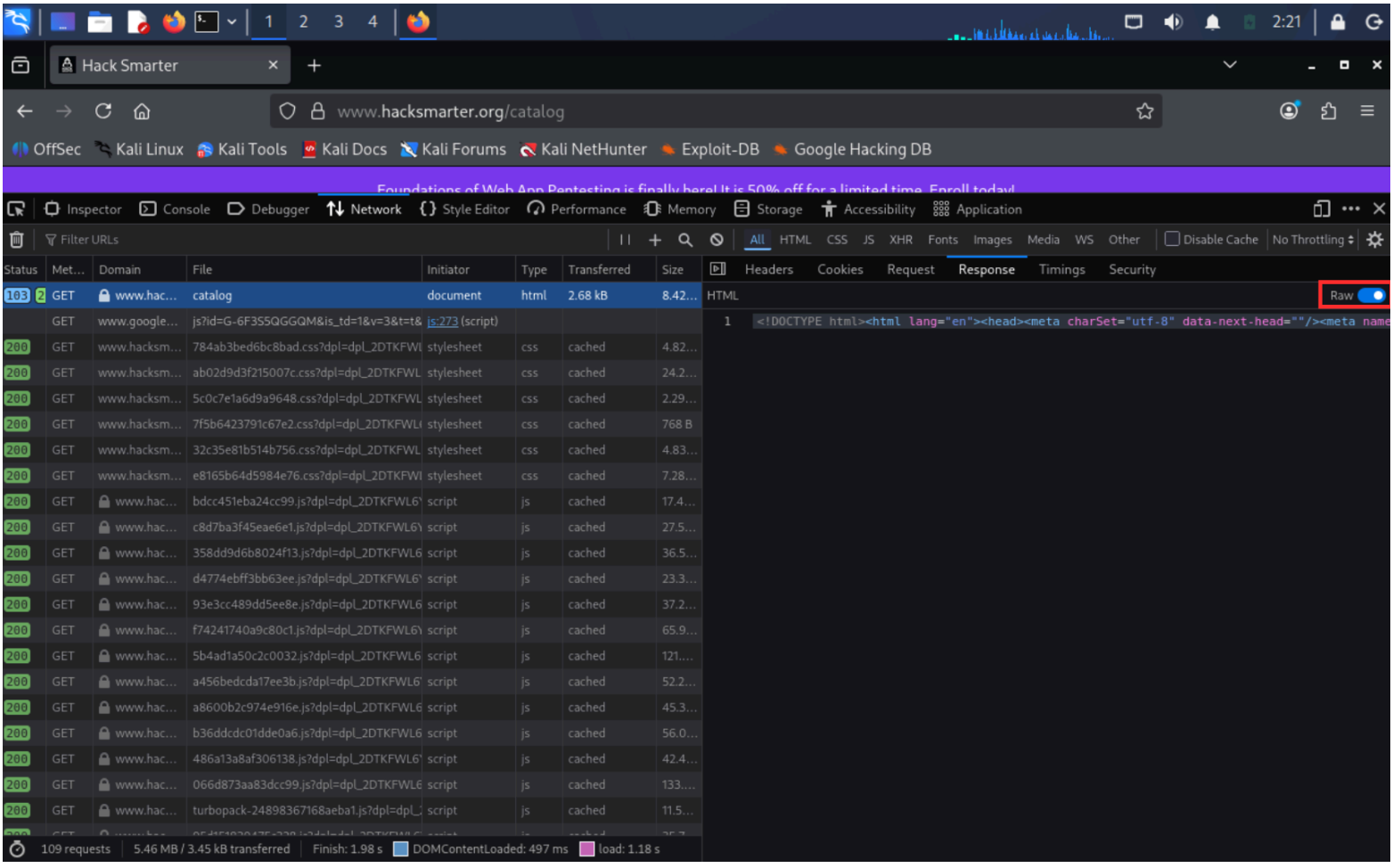
RESPONSE (SERVER → CLIENT)

Server sends:

Response (Visible in the “Response Tab”)



We can also see the: **`HTML code`** by clicking on the raw.



RENDERING (IMPORTANT CONCEPT)

Browser takes HTML and converts it into:

- Buttons
- Images
- UI

That's why you DON'T see raw HTML

SIMPLE ANALOGY

- Request → "Give me page"
- Response → "Here is code"
- Browser → "Let me make it look pretty"

BIG MISTAKE (THIS WILL HOLD YOU BACK)

Wrong Mindset (CTF Mindset)

- Fast
- Rush
- Find flag quickly

Correct Mindset (Real Pentesting)

- Slow down
- Observe everything
- Be curious

Why?

Because:
Each request = possible vulnerability

Reality

If you skip requests:

- You WILL miss bugs
- You WILL fail in real pentests

ATTACK SURFACE (VERY IMPORTANT)

One page = MANY entry points

Each request can be:

- manipulated
- tested
- attacked

Think like this:

Instead of:

> “Open website → find bug”

Think:

> “Inspect every request → find weakness”

BURP SUITE (THE REAL WEAPON)

Now comes the important part.

WHERE BURP SITS

Flow becomes:

```
Browser → Burp → Server
Server → Burp → Browser
```

Burp is in the **middle**

WHAT BURP CAN DO

1. Intercept (STOP request)

Pause request before it reaches server

2. Modify (EDIT request)

Change:

- data
- headers
- parameters

3. Forward (SEND request)

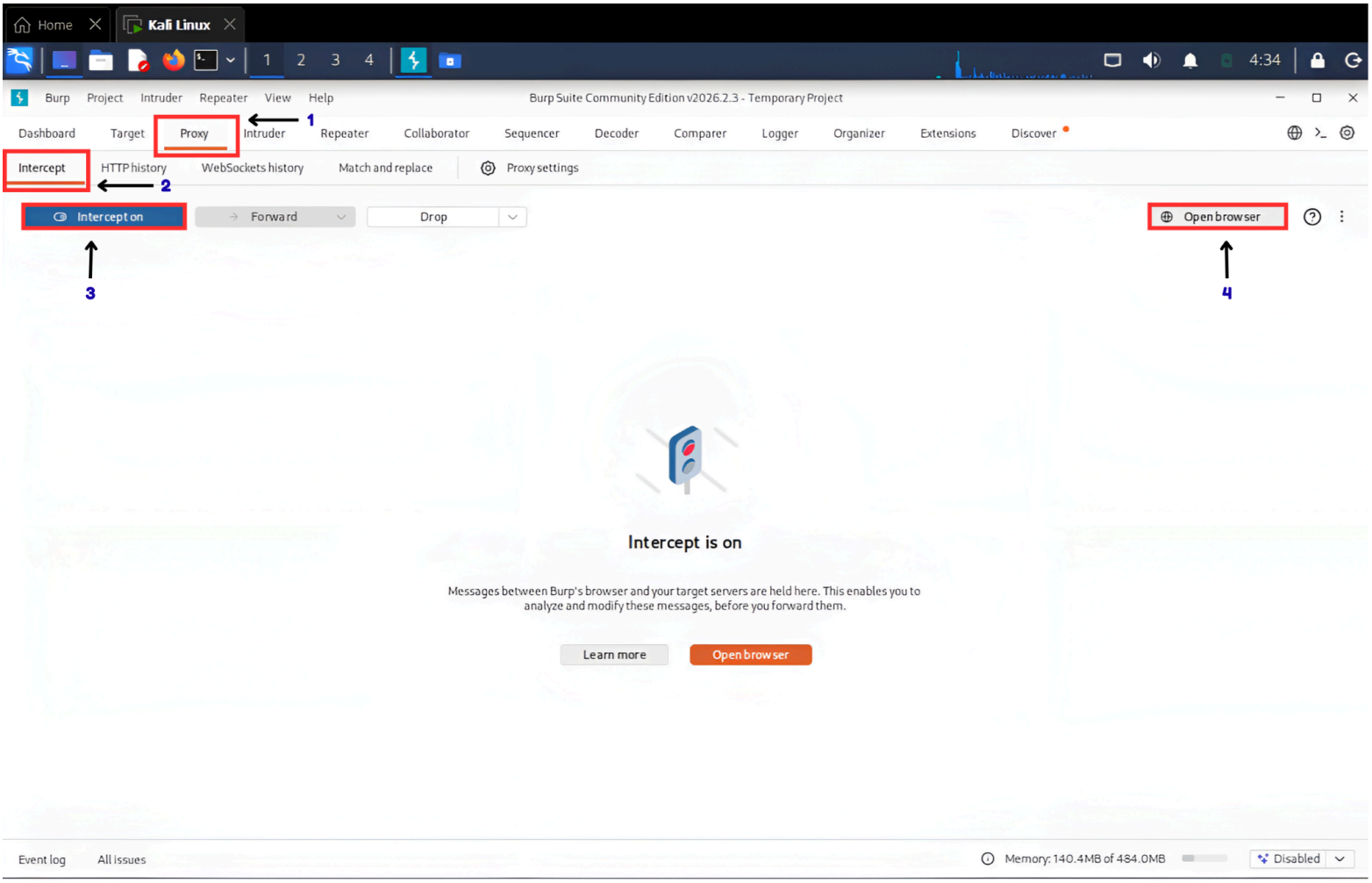
Send modified request to server

4. Intercept Response

See response BEFORE browser sees it

BURP DEMO FLOW

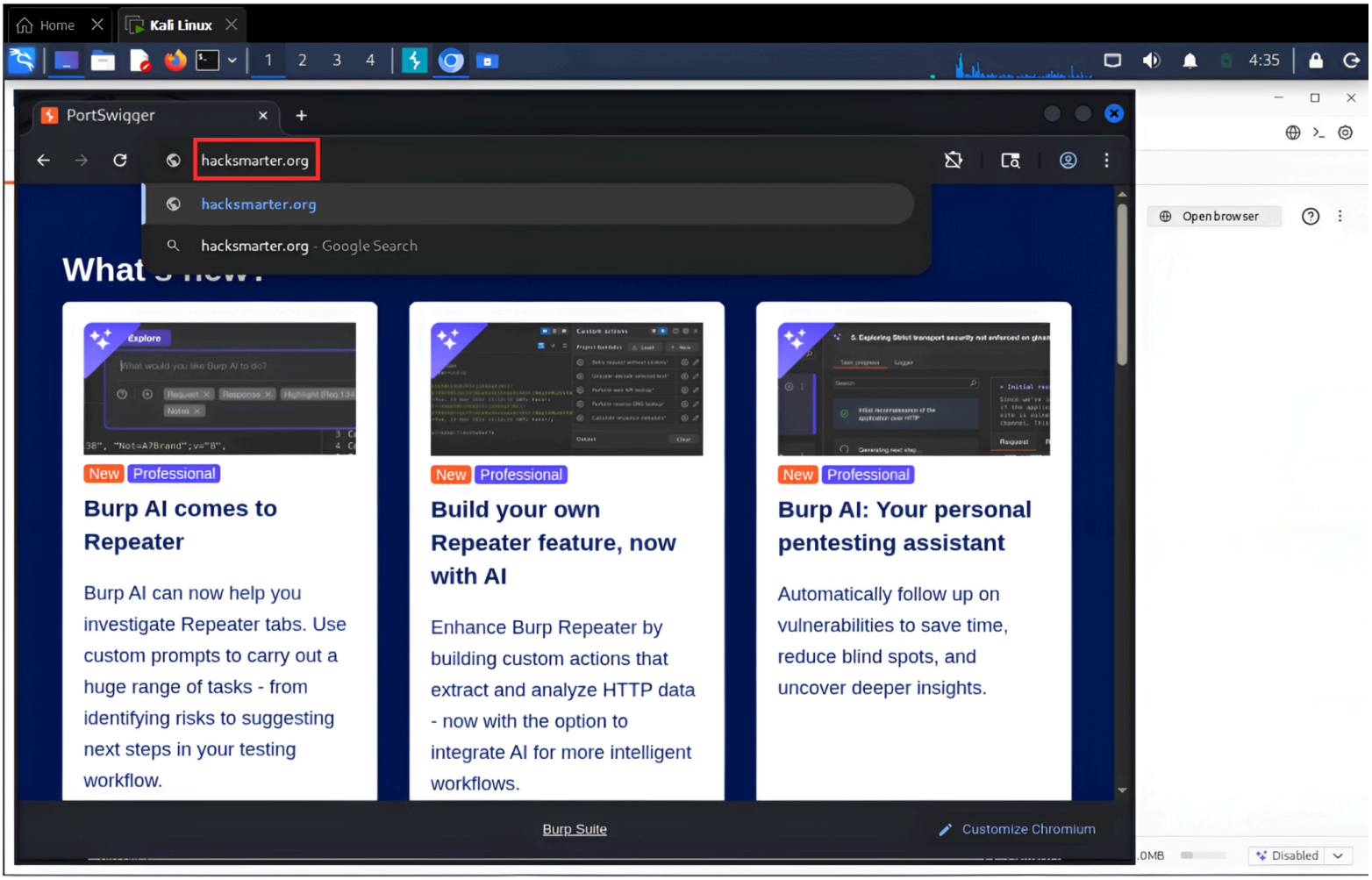
Step 1: Turn ON Intercept



Step 2: Open Website

Browser sends request → Burp catches it

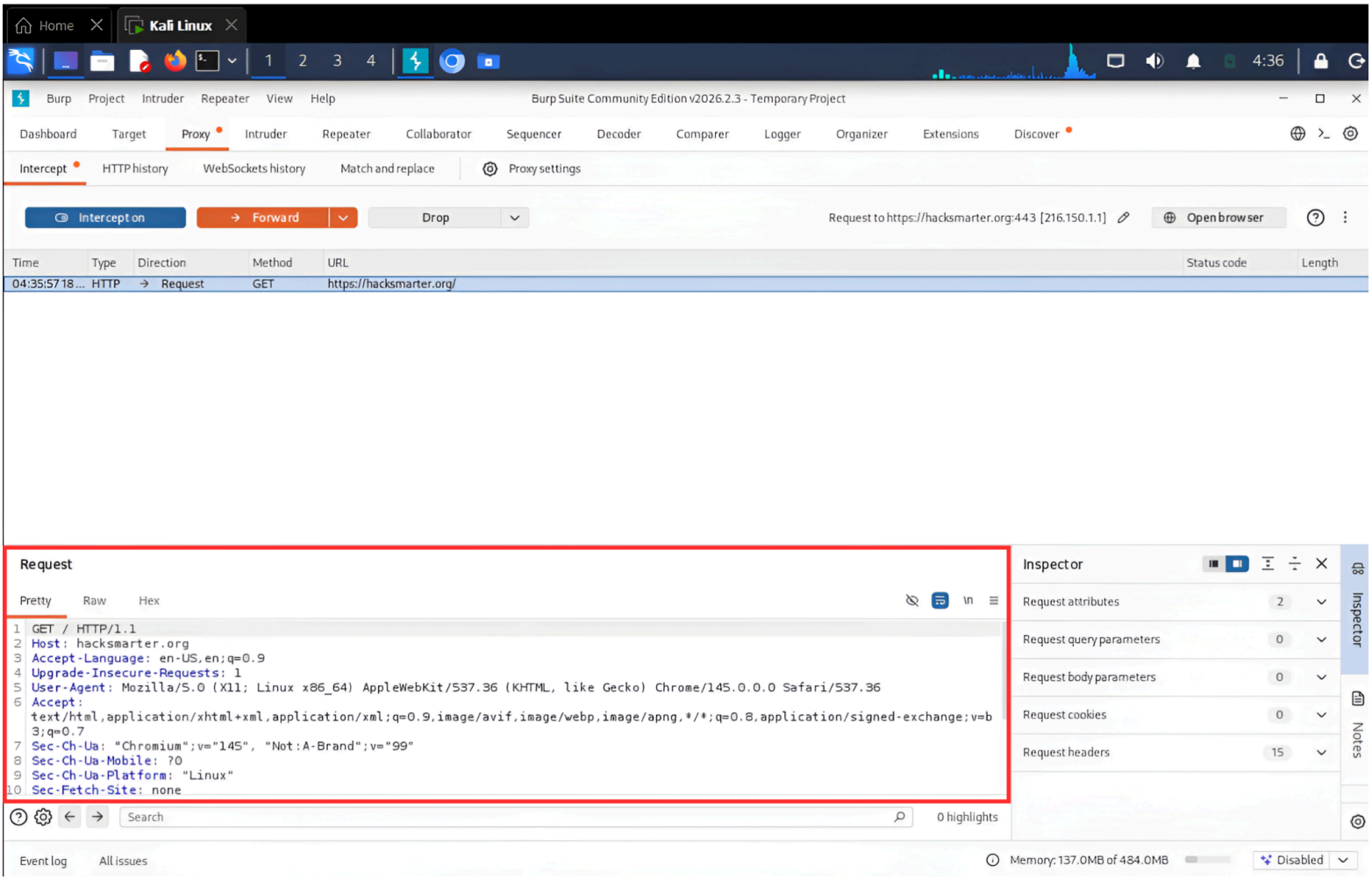
I am now going to search hacksmarter.org in my browser because that is where I am learning this course from and as soon as I press enter, the first request gets captured (There are going to be a bunch of other request shown after I allow this one to reach the server first by clicking the forward button)



Step 3: Inspect Request

You'll see:

```
GET / HTTP/1.1
Host: hacksmarter.org
```



Step 4: Modify (Optional)

You can:

- change values
- inject payloads

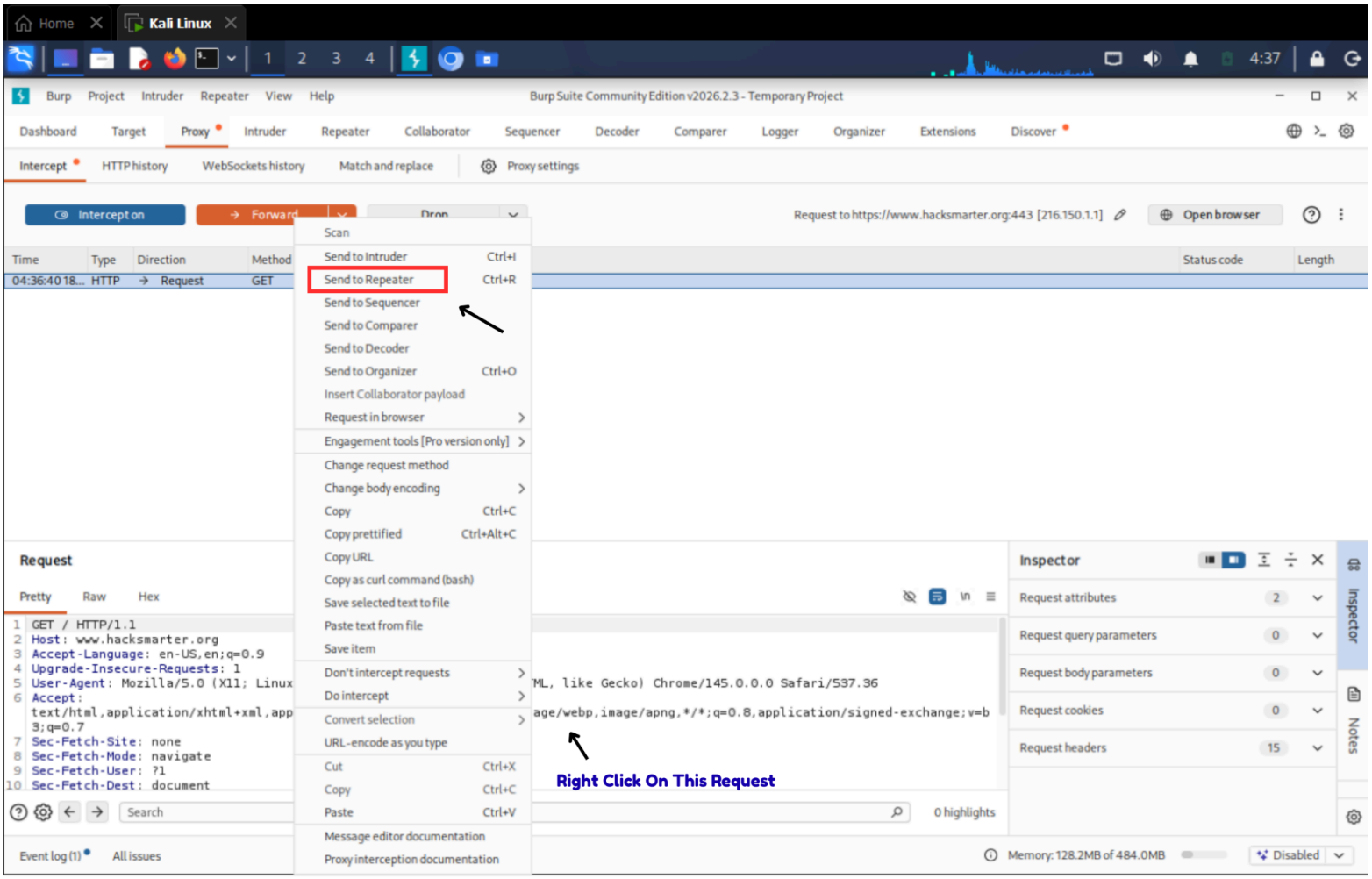
Step 5: Forward Request

Click **Forward** to forward this request to the server.

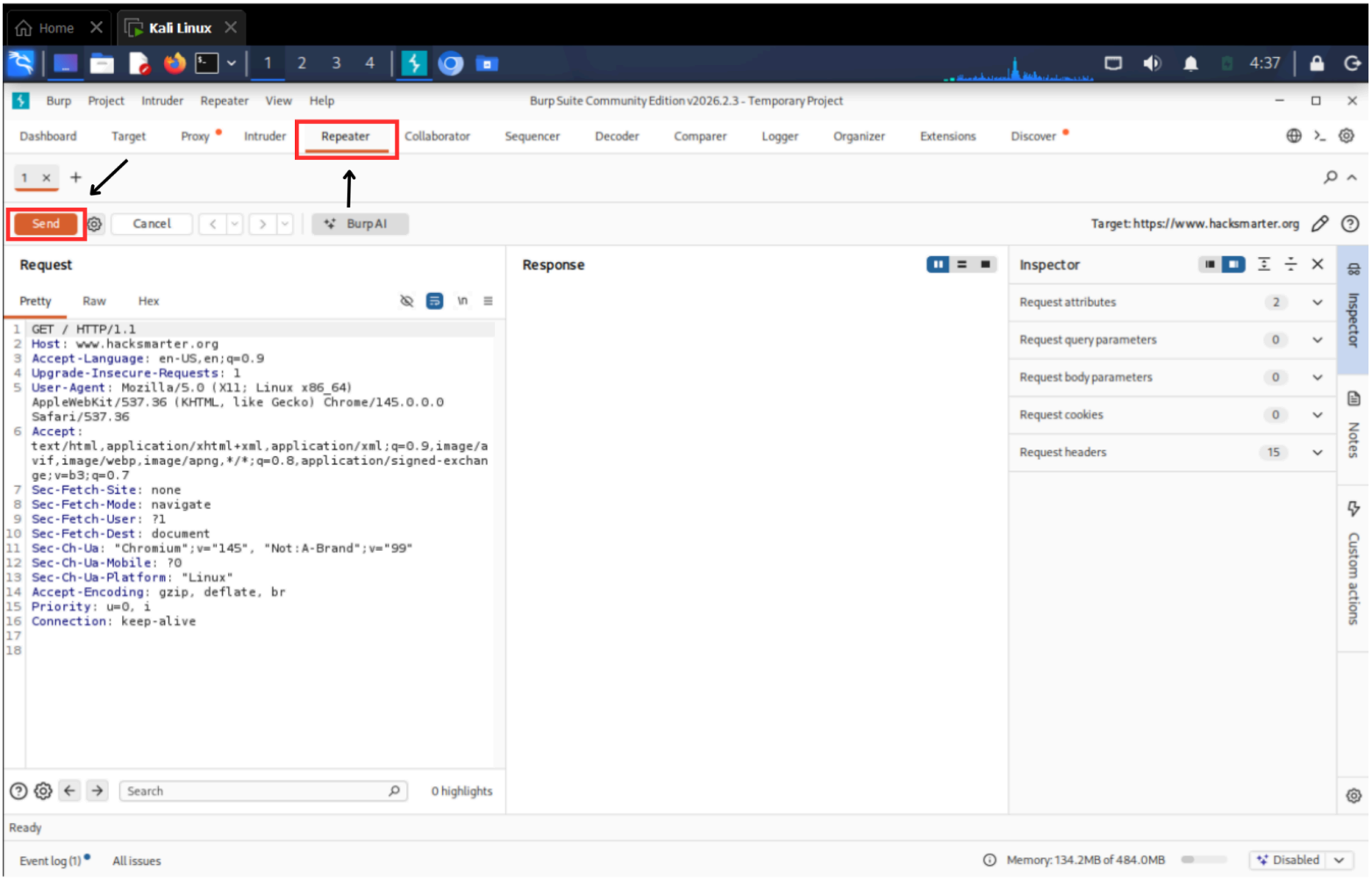
Step 6: View Response

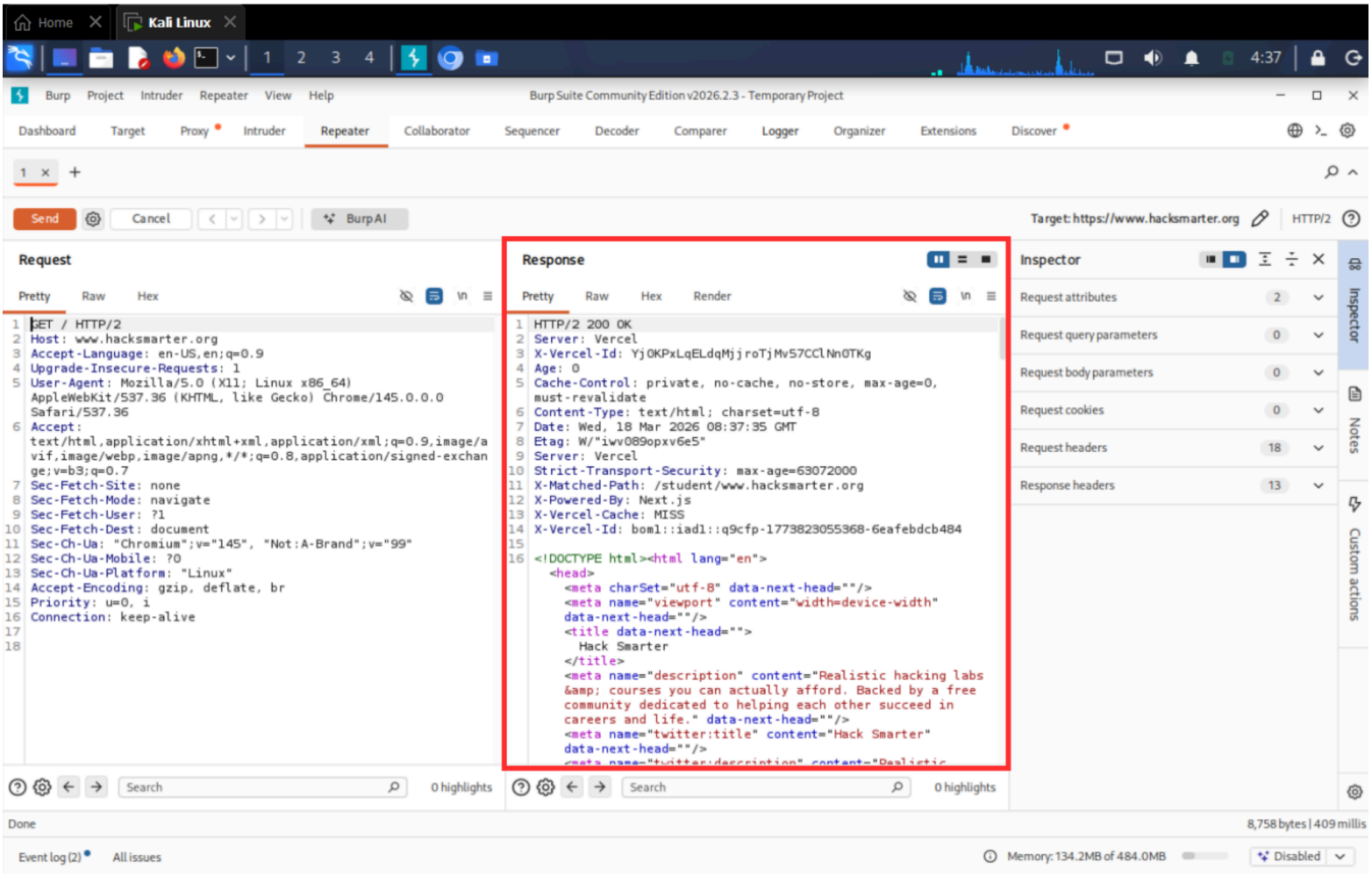
Server responds → Burp shows it

So we can use **Forward** to forward the request which I did here after which in the below image there is another request captured, and before I let it forward I can see its response by Right clicking on the request of which i wanna see the response and then clicking on the “Send to Repeater” and then click on the “Repeater” tab to see this request there.

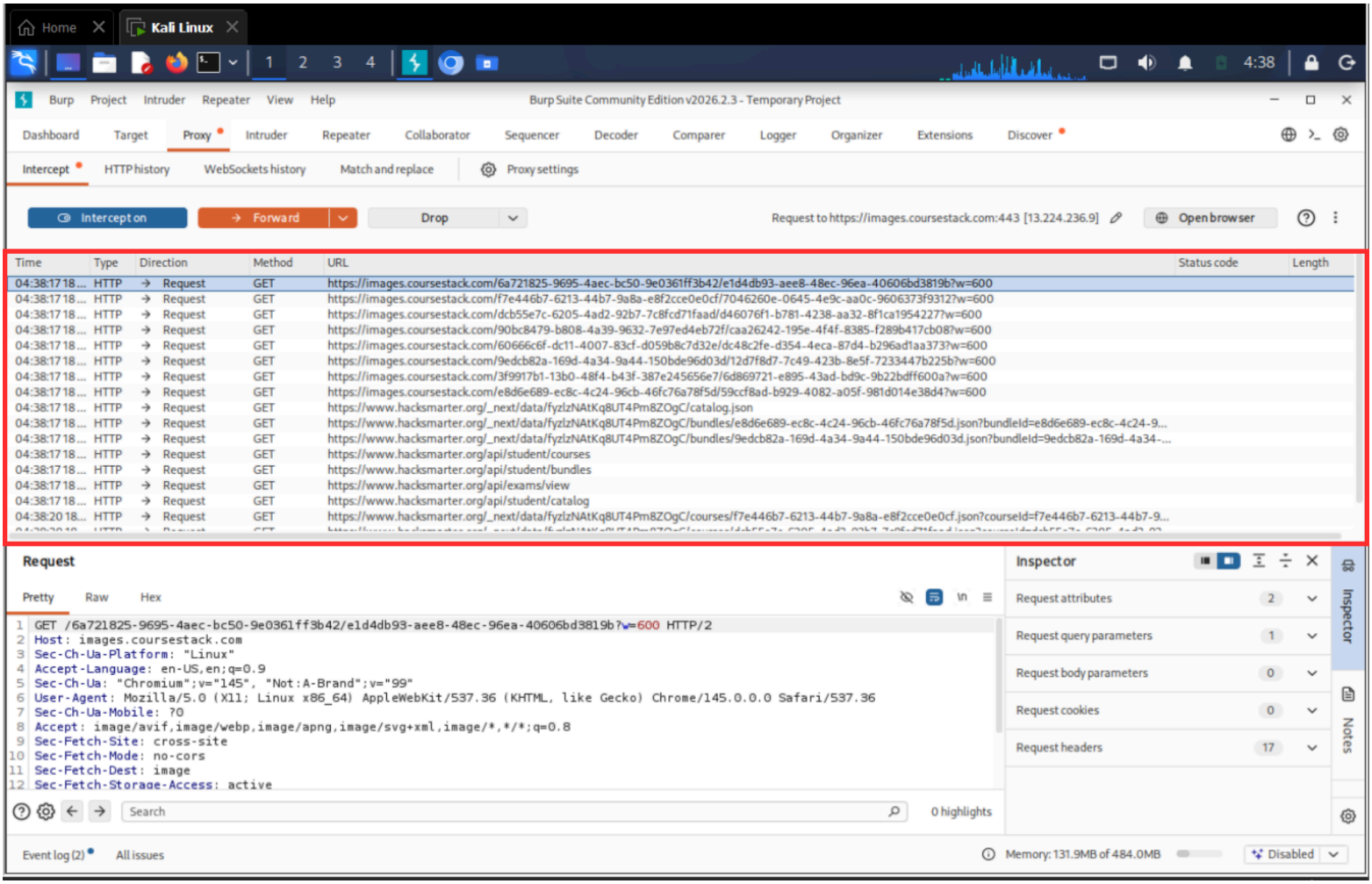


To see the response of this particular request just select the “Send” and the response to this request appears.

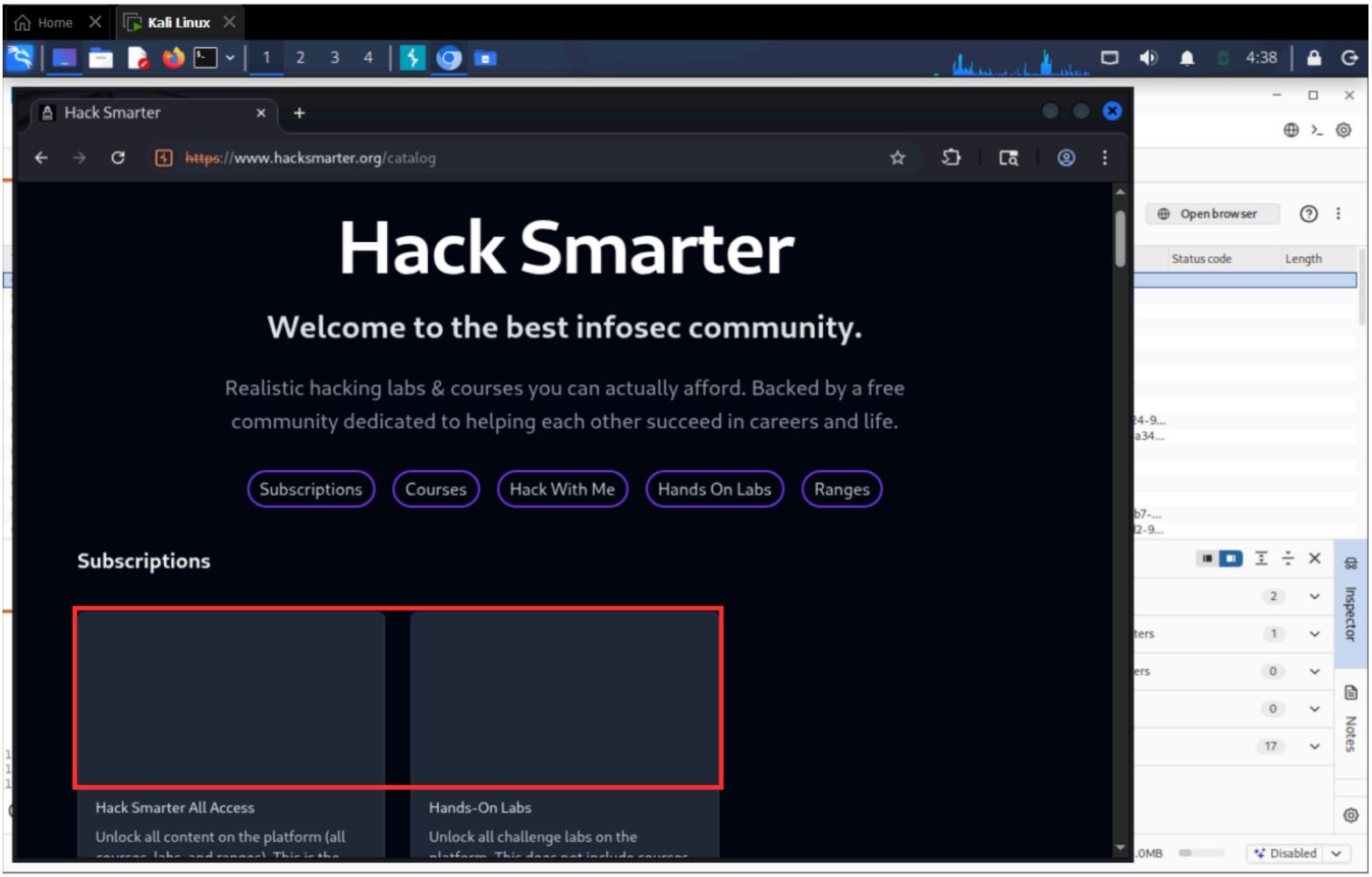

Home
Kali Linux
4:37
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.2.3 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Send
Cancel
BurpAI
Target: https://www.hacksmarter.org
Request
Pretty Raw Hex
GET / HTTP/1.1
Host: www.hacksmarter.org
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Linux"
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
Response
Inspector
Request attributes 2
Request query parameters 0
Request body parameters 0
Request cookies 0
Request headers 15
Search 0 highlights
Ready
Event log (1) All issues
Memory: 134.2MB of 484.0MB
Disabled


Home
Kali Linux
4:37
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.2.3 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Send
Cancel
BurpAI
Target: https://www.hacksmarter.org
HTTP/2
Request
Pretty Raw Hex
GET / HTTP/2
Host: www.hacksmarter.org
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Linux"
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
Response
Pretty Raw Hex Render
HTTP/2 200 OK
Server: Vercel
X-Vercel-Id: YjOKPxLqELdqMjjroTjMv57CClNn0TKg
Age: 0
Cache-Control: private, no-cache, no-store, max-age=0, must-revalidate
Content-Type: text/html; charset=utf-8
Date: Wed, 18 Mar 2026 08:37:35 GMT
Etag: W/"iwv089opxv6e5"
Server: Vercel
Strict-Transport-Security: max-age=63072000
X-Matched-Path: /student/www.hacksmarter.org
X-Powered-By: Next.js
X-Vercel-Cache: MISS
X-Vercel-Id: bom1::iad1::q9cfp-1773823055368-6eafebdcb484
<!DOCTYPE html><html lang="en">
<head>
<meta charSet="utf-8" data-next-head=""/>
<meta name="viewport" content="width=device-width" data-next-head=""/>
<title data-next-head="">
Hack Smarter
</title>
<meta name="description" content="Realistic hacking labs & courses you can actually afford. Backed by a free community dedicated to helping each other succeed in careers and life." data-next-head=""/>
<meta name="twitter:title" content="Hack Smarter" data-next-head=""/>
Inspector
Request attributes 2
Request query parameters 0
Request body parameters 0
Request cookies 0
Request headers 18
Response headers 13
Search 0 highlights
Search 0 highlights
Done
8,758 bytes | 409 millis
Event log (2) All issues
Memory: 134.2MB of 484.0MB
Disabled

We can see a bunch of other requests are also captured now.



Now previously I forwarded some requests but still the website will not load fully until I forward these requests as well.



As we can see the images aren't fully loaded yet.

IMPORTANT DIFFERENCE

Tool	What it does
Browser DevTools	Shows what happened
Burp Suite	Lets you CONTROL what happens

WHY IS THIS IMPORTANT

If you only use browser:
You are just observing

If you use Burp:
You are attacking

FINAL CORE UNDERSTANDING

Web app = collection of requests

Pentesting = analyzing & breaking those requests

SIMPLE SUMMARY

- Website loads → MANY requests
- Each request = opportunity
- Browser shows requests
- Burp controls requests
- Real pentesting = slow + deep analysis

If you:

- rush
- don’t inspect requests
- ignore details

You will stay a beginner forever
